

Computer Vision Assignment Report

This report documents the implementation and analysis of two computer vision experiments conducted using Python and OpenCV. The first notebook focuses on lane detection from road images, while the second explores foundational image processing techniques such as Gaussian filtering, edge detection, contour extraction, and morphological transformations. Each experiment is supported by code explanations and representative results from test images, highlighting the visual effects and algorithmic accuracy of the methods employed.

The main objective of these experiments is to understand how low-level computer vision algorithms transform raw images into structured data that can be interpreted by machines. Classical vision methods like edge detection, thresholding, and region masking serve as the foundation for modern autonomous vision systems. Two experiments were implemented: (1) lane detection using Canny edge detection and region-of-interest masking; and (2) general image processing operations to enhance and extract features from sample images.

Methodology and Code Explanation

The implementation was conducted in two Jupyter notebooks. Each notebook employed OpenCV and NumPy for efficient image manipulation and processing. The following subsections describe the main code components and their functions.

1. Lane Detection (vision-assignment-1-part-2.ipynb)

The core function defined in this notebook is `detect_lanes(img)`. It performs the following steps: 1. Converts the input RGB frame into grayscale to reduce complexity. 2. Applies Gaussian blur to suppress noise and smooth edges. 3. Uses the Canny edge detector to extract high-contrast boundaries corresponding to road lines. 4. Defines a region of interest (ROI) mask to isolate the bottom triangular section of the image where lanes are expected. 5. Combines the ROI with the Canny edges to retain only the relevant edge features. 6. Applies the Hough Transform to detect line segments corresponding to lanes. 7. Overlays the detected lines on the original image to visualize the final lane detection result.

Each step of the code was modularized to allow individual visualization. For example, intermediate outputs such as grayscale conversion, edge maps, and masked edges were displayed using `imshow()` calls. The `show()` helper function facilitated organized visualization of the pipeline output.

Representative Results:

Figure 1 shows the grayscale image, demonstrating loss of color but retention of structure. Figure 2 presents the Canny edge map highlighting road lines and object boundaries. Figure 3 depicts the masked region of interest, isolating the lower triangular portion of the image. Finally, Figure 4 overlays detected lane lines on the original road image, showing successful localization of left and right lane markers under good lighting conditions.

2. Image Processing Operations (cv-asg-1.ipynb)

The second notebook extends classical vision tasks to multiple image processing techniques. The main steps implemented are: 1. **Grayscale Conversion:** The RGB image is converted into a single-channel grayscale representation to simplify further analysis. 2. **Gaussian Filtering:** A Gaussian kernel is applied to reduce high-frequency noise and smooth image details. 3. **Edge**

Detection:** Both Canny and Sobel operators are implemented to visualize gradient intensity and direction. 4. **Thresholding:** Pixels are segmented into binary regions based on intensity to enhance object boundaries. 5. **Contour Detection:** OpenCV's `findContours()` function is used to extract geometric outlines of shapes in the image. 6. **Morphological Transformations:** Dilation and erosion are applied to refine detected shapes and suppress small noise.

Representative Results:

Figure 5 shows the grayscale version of a sample test image, which preserves intensity details suitable for edge detection. Figure 6 displays the Gaussian-smoothed output where noise is reduced while retaining major structures. Figure 7 illustrates the Canny edge map highlighting object boundaries. Figure 8 demonstrates contour extraction, and Figure 9 shows the effects of dilation and erosion where objects become thicker or thinner depending on kernel size.

Discussion

The experimental results confirm that the implemented algorithms correctly transform image data according to theoretical expectations. Canny edge detection proved effective in isolating lane and object boundaries, while Gaussian smoothing improved edge stability. However, lane detection exhibited limitations under poor lighting or sharp curvature, suggesting the potential for adaptive thresholding and perspective correction. Similarly, morphological operations provided strong noise suppression but required parameter tuning to prevent information loss.

Conclusion

This study demonstrates the successful application of classical computer vision algorithms for lane detection and general image processing. The modular Python implementation allows clear visualization of each processing step and its impact on image interpretation. The experiments establish a foundation for advanced vision tasks such as object detection, segmentation, and real-time autonomous navigation. Future extensions may include deep learning-based approaches to enhance robustness under varying environmental conditions.